Gemini

```
import cv2
import numpy as np
import matplotlib.pyplot as plt # Import matplotlib

# ------------------------------------------------
# Direct Linear Transformation (DLT)
# ------------------------------------------------

def dlt_homography(src_points, dst_points):
    A = []

    for (x, y), (u, v) in zip(src_points, dst_points)
        A.append([-x, -y, -1, 0, 0, 0, u*x, u*y, u])
        A.append([0, 0, 0, -x, -y, -1, v*x, v*y, v])

    A = np.array(A)

    # Solve Ah = 0 using Singular Value Decomposition
    U, S, Vt = np.linalg.svd(A)

    # Last row of Vt gives the solution
    h = Vt[-1]

    # Convert into 3x3 matrix
    H = h.reshape(3, 3)

    # Normalize
    H = H / H[2, 2]

    return H


# ------------------------------------------------
# Read input image
#
```

```
#

image_path = "/content/wp3892792.jpg" # Changed to an
image = cv2.imread(image_path)

if image is None:
    print(f"Error: Could not load image from {image_pa
else:
    # ------------------------------------------------
    # Source points
    # Points selected from the original image
    # ------------------------------------------------

    src_points = np.array([
        [100, 100],
        [500, 100],
        [500, 400],
        [100, 400]
    ], dtype=np.float64)



    # ------------------------------------------------
    # Destination points
    # Points where the source points should move
    # ------------------------------------------------

    dst_points = np.array([
        [50, 50],
        [550, 80],
        [520, 450],
        [80, 400]
    ], dtype=np.float64)



    # ------------------------------------------------
    # Calculate Homography Matrix using DLT
    # ------------------------------------------------
```

```
H = dlt_homography(src_points, dst_points)

print("Homography Matrix:")
print(H)


# ------------------------------------------------
# Apply transformation
# ------------------------------------------------

height, width = image.shape[:2]

transformed_image = cv2.warpPerspective(
    image,
    H,
    (width, height)
)


# ------------------------------------------------
# Display results (using Matplotlib for Colab comp
# ------------------------------------------------

# Convert BGR to RGB for Matplotlib display
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
transformed_image_rgb = cv2.cvtColor(transformed_i

plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1)
plt.imshow(image_rgb)
plt.title('Original Image')
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(transformed_image_rgb)
plt.title('DLT Transformed Image')
plt.axis('off')
plt.show()
```

```
# ------------------------------------------------
# Save result
# ------------------------------------------------

cv2.imwrite("transformed.jpg", transformed_image)

print("Transformation completed successfully!")
print("Output saved as transformed.jpg")
```

```
Homography Matrix:
[[ 1.18181818e+00  1.38644174e-01 -8.07185255e+01]
 [ 6.22461731e-02  1.37759034e+00 -9.26559409e+01]
 [-1.84317401e-04  4.49859419e-04  1.00000000e+00]]
```

Original Image

DLT Transformed Image